

Preparación Laboratorio 9 Python 3

Introducción a las Listas en Python

Objetivo

Aprender qué son las listas en Python, cómo crear y modificar listas, acceder a sus elementos y recorrerlas usando for.

¿Qué es una lista?

Una lista es una colección ordenada de elementos. Es muy parecida al arreglo de numpy en cuanto a que es una colección de elementos, pero tiene ventajas y desventajas. Las ventajas se refieren a que es una estructura mucho más flexible, se pueden agregar y eliminar elementos. Las desventajas es que es más lento operar con ellas. Tampoco permiten operar matemáticamente con ellas como lo hacíamos con los arreglos. Para operar sobre sus elementos hay que hacerlos de a uno por uno, veamos:

Ejemplo:

```
numeros = [3, 5, 7, 9]
```

La lista puede guardar números, texto o valores mezclados.

```
datos = [10, "hola", 3.5]
```

Acceder a elementos

Cada elemento tiene una posición llamada índice.

```
frutas = ["manzana", "pera", "uva"]
```

Índice 0 → manzana

Índice 1 → pera

Índice 2 → uva

```
print(frutas[0])  
print(frutas[2])
```

Modificar listas

Cambiar valor de un elemento:

```
frutas[1] = "naranja"
print(frutas)
```

Agregar elementos:

```
frutas.append("kiwi")
print(frutas)
```

Largo de una lista

```
print(len(frutas))
```

len() indica cuántos elementos tiene la lista.

Recorrer listas con for

La estructura for permite recorrer los elementos de una lista.

```
numeros = [2, 4, 6, 8]
```

```
for n in range(len(numeros)):
    print(numeros[n])
```

range(len(numeros)) genera un rango de numero desde 0 hasta len(números)-1. La instrucción for hace que la variable (en este caso n, pero podríamos usar cualquier nombre) tome los valores 0 primero y ejecute las instrucciones subordinadas al for. Luego hace lo mismo con 1, 2, y 3 (len(numeros) en este caso es 4).

Ejercicios:

Realice las actividades que se mencionan en seguida y anote las respuestas a las preguntas en un archivo latex, Word o de texto simple. Suba este archivo a u-cursos como entrega de la preparación de laboratorio 9

Abra una ventana de IDLE de Python e ingrese y ejecute las siguientes instrucciones:

```
a = [1, 2, 3]
b = [4, 5, 6]
```

```
print(a+b)
```

```
print(a*2)
```

¿Cuál es la diferencia con hacer estas mismas instrucciones con arreglos de numpy? ¿Que se hubiese obtenido si a y b fueran arreglos de numpy?

Ejecute la instrucción `print(a*b)` y explique qué obtiene como respuesta.

Ejecute el siguiente código:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = [0,0,0]
d = [0,0,0]
for i in range(len(a)):
    c[i] = a[i] * b[i]
    d[i] = a[i]+b[i]
print(c)
print(d)
```

Luego el siguiente:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = []
d = []
for i in range(len(a)):
    c.append(a[i] * b[i])
    d.append(a[i]+b[i])
print(c)
print(d)
```

Explique en palabras que hacen ambos trozos de programa y por qué imprimen lo mismo

Desafío opcional:

Usar `append()` para construir listas `a` y `b` cada una con 3 valores ingresados por el usuario.